

TrustBank CBS — Platform Architecture

A secure, modern, modular core banking platform.

Prepared for institutions evaluating TrustBank CBS. This document describes how the platform is built, why it is dependable, and how it adapts to your institution's needs.

1. Executive Summary

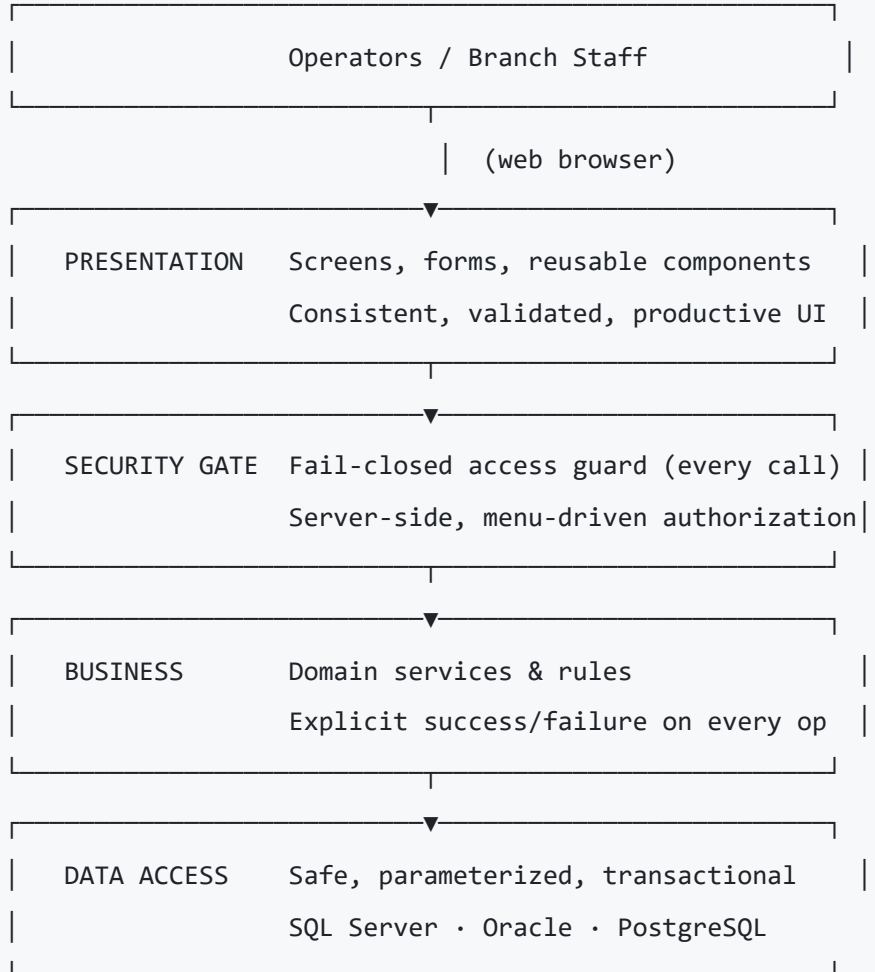
TrustBank CBS is a **core banking platform** built on **.NET 10**, the current long-term technology foundation from Microsoft. It is engineered around the priorities that matter most to a financial institution:

- **Security first.** Access is denied by default and granted only where explicitly authorized — on **every** request, enforced on the server, never trusted to the browser.
- **Correctness over convenience.** Every business operation reports an explicit success or failure. There are no silent errors, and multi-step money movements either complete fully or not at all.
- **Your database, your choice.** The same platform runs on **Microsoft SQL Server, Oracle, or PostgreSQL** — no vendor lock-in, no separate codebase per database.
- **Modular by design.** The system is organized into clearly bounded business capabilities (core banking, retail banking, accounts, lockers, clearing, HR), so it is simple to operate today and able to grow without a rewrite.
- **Consistent, productive experience.** A unified toolkit of screen components gives every operator a familiar, fast, error-resistant workflow.

The result is a platform that is **dependable to run, safe to operate, and economical to evolve** — designed for the realities of banking rather than retrofitted to them.

2. Architecture at a Glance

The platform is organized in clear layers, each with a single responsibility:

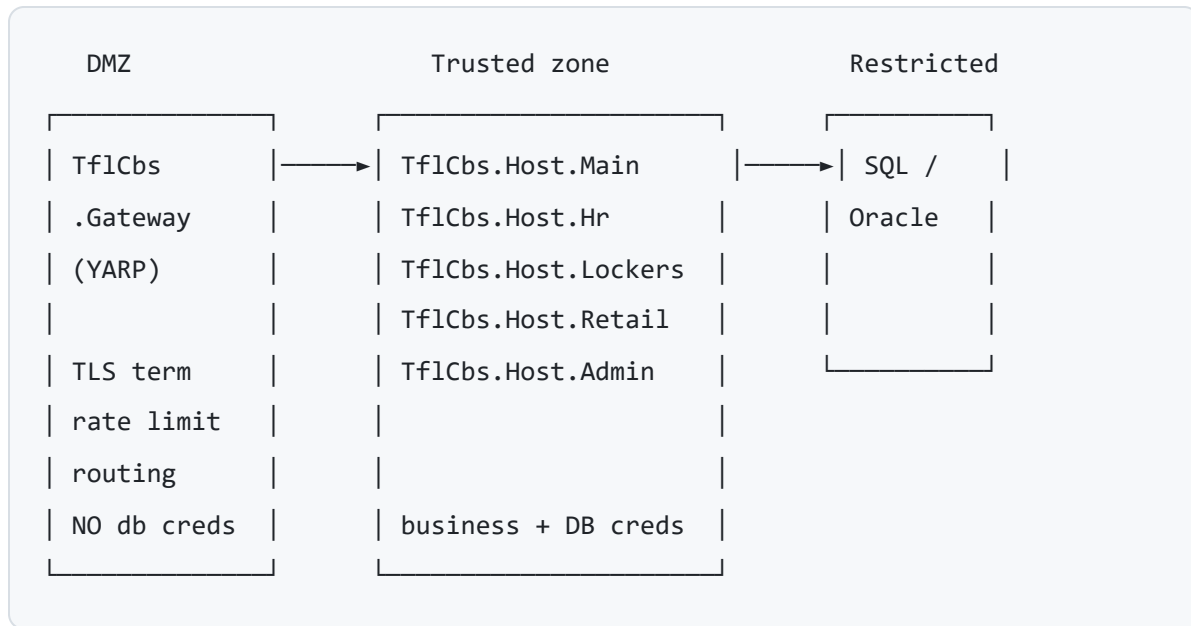


What happens on a single request, in plain terms: an operator opens a screen or submits a form → the platform confirms they are logged in, have selected their working module, and are authorized for that exact screen → the relevant business rules run and report a clear result → any data changes are written safely within a transaction → the operator receives a clear confirmation or a clear, actionable error. Every step is deliberate; nothing is assumed.

2.1 Deployment tiers (web tier / app tier / database tier)

The layers above describe the *code*. Deployment maps those layers onto **network security zones**, which is what an infrastructure or information-security review is

usually asking about when it asks "what runs on the web server and what runs on the application server?"



- **Web tier (DMZ)** — the reverse proxy / gateway. TLS termination, rate limiting, routing, and a place to sit a WAF. It holds **no business logic and no database connectivity**, so a compromise of the internet-facing box does not yield database credentials.
- **App tier (trusted zone)** — the application hosts. Business logic and the **only** database credentials in the estate. Reachable only from the web tier.
- **Database tier (restricted zone)** — reachable only from the app tier.

Two points that commonly come up:

- **This is a network-zoning question, not a product question.** "Application server" originates from the Java container era (WebLogic / WebSphere / JBoss); .NET has no equivalent product — the host process *is* the application server. Confirm whether the requirement is a zone or a named product before answering.
- **A "frontend on the web server, API on the app server" split requires a single-page application**, where the frontend is static files. The migrated screens are server-rendered, so presentation and business logic share a process by design; the zoning requirement is met by the gateway instead.

The Docker multi-host shape already implements this: only the gateway publishes a port and all backends are internal-only. The single-host shape places the application

directly in the app tier and expects an existing reverse proxy in front where the zoning is mandated.

3. Guiding Principles

These principles are not aspirations — they are enforced in the platform's structure and verified automatically.

Principle	What it means for your institution
Fail-closed security	Nothing is accessible unless explicitly permitted. A missing or unknown permission results in denial, not exposure.
Explicit outcomes	Every operation returns a clear success or failure with a traceable reference — no operation fails quietly.
All-or-nothing writes	Multi-step changes run inside a transaction; a partial failure rolls everything back, protecting balance integrity.
Database independence	One codebase runs on SQL Server, Oracle, or PostgreSQL — your infrastructure choice, not ours.
Modular boundaries	Business domains are separated by enforced internal boundaries, keeping the system maintainable as it grows.
Safe, additive change	New features and components extend the platform without breaking existing screens and workflows.

4. Business Capability Modules

The platform is divided into **bounded business capabilities**. Each owns its own rules and data access, and communicates with others only through well-defined contracts — so a change in one capability cannot destabilize another.

Capability	Scope
Core Banking	Menu and navigation, institutional parameters, broadcast and alerts, and the maker-checker authorization scroll — the platform services every other capability depends on.
Reference Data	Bank-wide master data with no domain behaviour: geography (state / district / taluka). Shared, so it is present in every deployment.
Authentication	Login, credential policy, password lifecycle, and session validation, including single-active-session enforcement.
Administration	Users, roles, module permissions, and oversight of currently connected users.
Accounts	Account servicing, account inquiry, holdings, business assessment.
Retail Banking	Customer-facing retail banking operations.
Lockers	Safe-deposit locker administration and servicing.
Clearing	Cheque clearing and settlement processing.
Human Resources	Staff and HR administration for the institution.

One platform, one deployment — with clean internal separation that keeps each capability independently understandable, testable, and evolvable. Each capability is a **separately built component**, and the deployment configuration decides which are present: the full institution, or a focused subset.

5. Security & Access Control

Security in TrustBank CBS is **structural**, not an add-on. The platform assumes a hostile environment and protects accordingly.

- **Fail-closed access guard on every request.** Before any screen or action runs, a central guard confirms the user is authenticated, has selected a working module, and is authorized for that specific screen. Anything unrecognized is denied.
 - **Server-side, menu-driven authorization.** What a user may access is decided on the server from their assigned permissions. The browser is never trusted to make or carry that decision, so it cannot be manipulated by the client.
 - **Tamper-proof record handling.** When a record is handed from a list to a form, it is referenced by an **encrypted, single-use token** rather than a raw identifier. This prevents tampering, prevents reuse, and blocks attempts to reach records the operator was never shown.
 - **Managed sessions with single-active-session enforcement.** Each user's session is tracked server-side; if a user is signed in elsewhere or force-logged-out, active sessions are invalidated immediately.
 - **Encryption of sensitive identifiers** and protected data, with key management handled by the platform.
 - **No credentials in code.** Connection details and secrets come from secured configuration — never hard-coded, never logged.
 - **Segregation-ready workflows.** The permission model supports separating who can initiate from who can authorize, aligning with banking control requirements.
-

6. Reliability & Data Integrity

Banking software is judged on whether the numbers are always right. The platform is built so they are.

- **Explicit success/failure on every operation.** Business operations return a structured result indicating success or failure, the reason, and a **correlation reference** for traceability. Failures cannot be silently ignored.
- **All-or-nothing transactions.** Operations that touch multiple records run as a single unit of work. If any step fails, the entire change is rolled back — there is no partial state.
- **Safe data access only.** All database access is parameterized, eliminating injection risks. New data operations use a controlled, typed access layer rather than ad-hoc queries.

- **Structured, traceable audit logging.** Outcomes are recorded with timestamps, severity, and correlation references in rolling daily logs — supporting investigation, audit, and regulatory inquiry.
- **Clear operator feedback.** Successful changes produce an explicit confirmation the operator must acknowledge; preventable mistakes are caught early with immediate, specific guidance.

7. Technology Foundation

Area	Choice	Why it matters
Platform	.NET 10 (current long-term-support generation)	Modern, supported, high-performance, with a long maintenance horizon.
Application model	ASP.NET Core MVC	A mature, well-understood, widely-supported web architecture.
Databases	SQL Server, Oracle, PostgreSQL	Run on your existing database investment — no lock-in, one codebase.
Composition	Modern dependency injection	Components are wired centrally and cleanly, keeping the system testable and maintainable.
Logging	Structured logging with daily rolling files, with sensitive-field masking applied centrally	Operational visibility and an audit-friendly record out of the box; personal identifiers are redacted before they reach a log file.
Observability	OpenTelemetry traces and metrics over the standard OTLP protocol	Plugs into whatever monitoring stack you already run — no proprietary agent. Off by

Area	Choice	Why it matters
		default; enabled by configuration.
Schema management	Versioned, journalled database migrations	Every schema change is applied exactly once, in order, and recorded — with a read-only <code>status</code> check for a change-advisory board.
Configuration	Externalized, secrets kept out of code	Safe, environment-specific deployment without code changes.

The platform builds on standard, mainstream Microsoft technology — meaning a wide talent pool, long-term supportability, and no dependence on niche or proprietary frameworks.

8. Modularity & Future-Readiness

TrustBank CBS uses a **modular-monolith** architecture — widely regarded as the soundest starting point for a core banking system.

Why this is the right model for banking:

- **Simple and reliable to operate today.** One application, one database, and strong transactional consistency — exactly what money movement requires. There is no distributed-systems complexity to manage or to get wrong.
- **Cleanly bounded inside.** Each business capability is separated by enforced boundaries, so the system stays maintainable and understandable even as it grows.
- **Ready to scale without a rewrite.** Because the boundaries are real and enforced, individual capabilities can be scaled out independently **if and when** transaction volumes justify it — an evolution, not a re-platforming.
- **Configurable footprint.** The same software can run the full bank or a single capability, governed by configuration — useful for phased rollouts, specialized

deployments, and testing.

This gives your institution the **operational simplicity of a monolith now** and the **flexibility of a modular system later**, on your timeline and your terms.

9. Consistent User Experience

Operator productivity and accuracy come from consistency. The platform ships with a **unified component toolkit** used across every screen:

- **Record and account pickers** — fast, validated selection of customers, accounts, and master data, with built-in search.
- **Search dialogs** — a consistent way to find records across the system.
- **Guided date selection** — structured date entry that prevents common input errors.
- **Standardized notifications** — clear, consistent confirmations for completed actions and immediate, specific guidance for mistakes.
- **Consistent action bars** — the same controls in the same place on every screen.

Because every screen is built from the same toolkit, operators learn the system once and apply that knowledge everywhere — reducing training time and operational error.

10. Quality Engineering

Quality is built into the development process, not inspected in afterwards.

Verification runs in **four independent tiers**, each catching what the others cannot:

Tier	What it proves
Cross-database business tests	The platform's business logic is verified automatically against SQL Server, Oracle, and PostgreSQL — so behaviour is consistent regardless of your database choice.

Tier	What it proves
Architecture boundary tests	Automated checks fail the build if a developer accidentally crosses a module or layer boundary — including a meta-check that verifies each boundary is still genuinely being enforced rather than passing vacuously.
In-process pipeline tests	The real application is started in-process and exercised over HTTP, so the security guard, the anti-forgery protection and authenticated access are proven in their true running order — not in a simulation of it.
Browser end-to-end tests	A real browser drives the live application, confirming what only a browser can see: the strict content-security policy actually being enforced, session and cookie behaviour, and every reusable screen component working.

- **Non-destructive, repeatable verification.** Tests are designed to run safely and consistently, independent of specific data.
- **Performance measured, not assumed.** Rewritten database logic is timed against the original it replaces, and the comparison is recorded as a dated report.

This means the architectural guarantees in this document are not just promises — they are continuously and automatically verified.

Appendix — Technical Overview

For the evaluating institution's architects and technical reviewers. This section summarizes the engineering structure behind the platform.

Layered structure

The platform separates **reusable foundation** from **business screens** at the assembly level:

- **Contracts layer** — a single dependency-free assembly holding the result contract, the module contract, and the interfaces (*ports*) through which the framework obtains business data. Everything else compiles against it, which is

what lets the framework and the business modules cooperate without either one referencing the other.

- **Framework layer** — web plumbing, security guard, session management, search infrastructure, and the reusable component models. Self-contained and reusable; it has **no dependency on screen code and no dependency on any business module** — both separations are compiler-enforced, not merely conventional, and are additionally covered by automated architecture tests.
- **Business/services layer** — domain services that encapsulate banking rules and data access. Each business capability is its **own assembly** with zero web-framework dependency, so it is testable and host-agnostic.
- **Presentation layer** — screens ship as self-contained component libraries, one per business capability, plus one shared library holding the common layout, partials and client assets. A host loads exactly the screen libraries its configuration enables.
- **Hosts** — thin composition roots. One full host runs the whole institution; optional per-capability hosts run a single capability behind a reverse-proxy gateway when independent deployment or scaling is wanted.
- **Entities** — typed representations of database tables, shared across the platform.

Composition and module boundaries

- Each business module exposes a single registration entry point that wires its own services; the application composes the full system by registering each module once, in one place.
- **Cross-module communication is contract-only.** A module depends on another module's published interface and data shapes — never on its internal implementation. This is enforced by automated architecture tests, keeping the modules genuinely decoupled and independently evolvable.
- Internal database representations are not exposed across module boundaries; modules exchange purpose-built data shapes, so each capability can evolve its internals freely.

Data access

- A typed repository model (`Repository<T>`) provides standard read/write operations without hand-written SQL for routine work.

- Joins, projections, and aggregations use **parameterized** queries — never string-concatenated SQL.
- Multi-step writes run through a **transactional unit-of-work**, guaranteeing all-or-nothing semantics.
- A provider-selection layer lets the identical codebase target SQL Server, Oracle, or PostgreSQL; database-dialect differences are handled centrally, not scattered through business code.

Explicit results

- Every service operation returns a `Result` / `Result<T>` envelope carrying success/failure status, an error reason, and a correlation reference. Exceptions are not thrown across the service boundary, and failures cannot be silently dropped by callers.

Verification strategy

- **Cross-provider functional tests** validate business behavior against each supported database.
- **Architecture boundary tests** enforce the layer and module separation described above, failing the build on any violation.

TrustBank CBS — engineered for the realities of banking: secure by default, correct by construction, and ready to grow.

Document reviewed against the platform as built on 21 August 2026.